

https://community.chocolatey.org/install.ps1

https://community.chocolatey.org/install.ps1 - Author not stated, community.chocolatey.org.

- Published: date not stated
- Original: <https://community.chocolatey.org/install.ps1>
- Preserved from: <https://community.chocolatey.org/install.ps1> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

```
<# .SYNOPSIS Downloads and installs Chocolatey on the local machine. .DESCRIPTION
Retrieves the Chocolatey nupkg for the latest or a specified version, and downloads and
installs the application to the local machine. .NOTES
=====
===== Copyright 2017 - 2020 Chocolatey Software, Inc, and the original
authors/contributors from ChocolateyGallery Copyright 2011 - 2017 RealDimensions
Software, LLC, and the original authors/contributors from ChocolateyGallery at
https://github.com/chocolatey/chocolatey.org Licensed under the Apache License, Version
2.0 (the "License"); you may not use this file except in compliance with the License. You
may obtain a copy of the License at http://www.apache.org/licenses/LICENSE-2.0 Unless
required by applicable law or agreed to in writing, software distributed under the License is
distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND,
either express or implied. See the License for the specific language governing permissions
and limitations under the License.
=====
===== Environment Variables, specified as $env:NAME in PowerShell.exe
and %NAME% in cmd.exe. For explicit proxy, please set $env:chocolateyProxyLocation and
optionally $env:chocolateyProxyUser and $env:chocolateyProxyPassword For an explicit
version of Chocolatey, please set $env:chocolateyVersion = 'versionnumber' To target a
different url for chocolatey.nupkg, please set $env:chocolateyDownloadUrl = 'full url to
nupkg file' NOTE: $env:chocolateyDownloadUrl does not work with
$env:chocolateyVersion. To use built-in compression instead of 7zip (requires additional
download), please set $env:chocolateyUseWindowsCompression = 'true' To bypass the use
of any proxy, please set $env:chocolateyIgnoreProxy = 'true' .LINK For organizational
```

deployments of Chocolatey, please see <https://docs.chocolatey.org/en-us/guides/organizations/organizational-deployment-guide> #>

```
[CmdletBinding(DefaultParameterSetName = 'Default')] param( # The URL to download
Chocolatey from. This defaults to the value of # $env:chocolateyDownloadUrl, if it is set,
and otherwise falls back to the # official Chocolatey community repository to download the
Chocolatey package. # Can be used for offline installation by providing a path to a
Chocolatey.nupkg. [Parameter(Mandatory = $false)] [string] $ChocolateyDownloadUrl =
$env:chocolateyDownloadUrl, # Specifies a target version of Chocolatey to install. By
default, the latest # stable version is installed. This will use the value in #
$env:chocolateyVersion by default, if that environment variable is present. # This
parameter is ignored if -ChocolateyDownloadUrl is set. [Parameter(Mandatory = $false)]
[string] $ChocolateyVersion = $env:chocolateyVersion, # If set, uses built-in Windows
decompression tools instead of 7zip when # unpacking the downloaded nupkg. This will be
set by default if # $env:chocolateyUseWindowsCompression is set to a value other than
'false' or '0'. # # This parameter will be ignored in PS 5+ in favour of using the # Expand-
Archive built in PowerShell cmdlet directly. [Parameter(Mandatory = $false)] [switch]
$UseNativeUnzip = $( $envVar = "$env:chocolateyUseWindowsCompression".Trim() $value
= $null if ([bool]::TryParse($envVar, [ref] $value)) { $value } elseif ([int]::TryParse($envVar,
[ref] $value)) { [bool]$value } else { [bool]$envVar } ), # If set, ignores any configured
proxy. This will override any proxy # environment variables or parameters. This will be set
by default if # $env:chocolateyIgnoreProxy is set to a value other than 'false' or '0'.
[Parameter(Mandatory = $false)] [switch] $IgnoreProxy = $( $envVar =
"$env:chocolateyIgnoreProxy".Trim() $value = $null if ([bool]::TryParse($envVar, [ref]
$value)) { $value } elseif ([int]::TryParse($envVar, [ref] $value)) { [bool]$value } else {
[bool]$envVar } ), # Specifies the proxy URL to use during the download. This will default
to # the value of $env:chocolateyProxyLocation, if any is set.
[Parameter(ParameterSetName = 'Proxy', Mandatory = $false)] [string] $ProxyUrl =
$env:chocolateyProxyLocation, # Specifies the credential to use for an authenticated
proxy. By default, a # proxy credential will be constructed from the
$env:chocolateyProxyUser and # $env:chocolateyProxyPassword environment variables, if
both are set. [Parameter(ParameterSetName = 'Proxy', Mandatory = $false)]
[System.Management.Automation.PSCredential] $ProxyCredential ) #region Functions
function Get-Downloader { <# .SYNOPSIS Gets a System.Net.WebClient that respects
relevant proxies to be used for downloading data. .DESCRIPTION Retrieves a WebClient
object that is pre-configured according to specified environment variables for any proxy
and authentication for the proxy. Proxy information may be omitted if the target URL is
considered to be bypassed by the proxy (originates from the local network.) .PARAMETER
Url Target URL that the WebClient will be querying. This URL is not queried by the function,
it is only a reference to determine if a proxy is needed. .EXAMPLE Get-Downloader -Url
$fileUrl Verifies whether any proxy configuration is needed, and/or whether $fileUrl is a
URL that would need to bypass the proxy, and then outputs the already-configured
WebClient object. #> [CmdletBinding()] param( [Parameter(Mandatory = $false)] [string]
$Url, [Parameter(Mandatory = $false)] [string] $ProxyUrl, [Parameter(Mandatory = $false)]
[System.Management.Automation.PSCredential] $ProxyCredential ) $downloader = New-
```

```

Object System.Net.WebClient $defaultCreds =
[System.Net.CredentialCache]::DefaultCredentials if ($defaultCreds) {
$downloader.Credentials = $defaultCreds } if ($ProxyUrl) { # Use explicitly set proxy.
Write-Host "Using explicit proxy server '$ProxyUrl'." $proxy = New-Object
System.Net.WebProxy -ArgumentList $ProxyUrl, <# bypassOnLocal: #> $true
$proxy.Credentials = if ($ProxyCredential) { $ProxyCredential.GetNetworkCredential() }
elseif ($defaultCreds) { $defaultCreds } else { Write-Warning "Default credentials were
null, and no explicitly set proxy credentials were found. Attempting backup method." (Get-
Credential).GetNetworkCredential() } if (-not $proxy.IsBypassed($Url)) { $downloader.Proxy
= $proxy } } else { Write-Host "Not using proxy." } $downloader } function Request-String
{ <# .SYNOPSIS Downloads content from a remote server as a string. .DESCRIPTION
Downloads target string content from a URL and outputs the resulting string. Any existing
proxy that may be in use will be utilised. .PARAMETER Url URL to download string data
from. .PARAMETER ProxyConfiguration A hashtable containing proxy parameters (ProxyUrl
and ProxyCredential) .EXAMPLE Request-String https://community.chocolatey.org/install.ps1
Retrieves the contents of the string data at the targeted URL and outputs it to the pipeline.
#> [CmdletBinding()] param( [Parameter(Mandatory = $true)] [string] $Url,
[Parameter(Mandatory = $false)] [hashtable] $ProxyConfiguration ) (Get-Downloader $url
@ProxyConfiguration).DownloadString($url) } function Request-File { <# .SYNOPSIS
Downloads a file from a given URL. .DESCRIPTION Downloads a target file from a URL to
the specified local path. Any existing proxy that may be in use will be utilised. .PARAMETER
Url URL of the file to download from the remote host. .PARAMETER File Local path for the
file to be downloaded to. .PARAMETER ProxyConfiguration A hashtable containing proxy
parameters (ProxyUrl and ProxyCredential) .EXAMPLE Request-File -Url
https://community.chocolatey.org/install.ps1 -File $targetFile Downloads the install.ps1
script to the path specified in $targetFile. #> [CmdletBinding()] param(
[Parameter(Mandatory = $false)] [string] $Url, [Parameter(Mandatory = $false)] [string]
$File, [Parameter(Mandatory = $false)] [hashtable] $ProxyConfiguration ) Write-Host
"Downloading $url to $file" (Get-Downloader $url @ProxyConfiguration).DownloadFile($url,
$file) } function Set-PSConsoleWriter { <# .SYNOPSIS Workaround for a bug in output
stream handling PS v2 or v3. .DESCRIPTION PowerShell v2/3 caches the output stream.
Then it throws errors due to the FileStream not being what is expected. Fixes "The OS
handle's position is not what FileStream expected. Do not use a handle simultaneously in
one FileStream and in Win32 code or another FileStream." error. .EXAMPLE Set-
PSConsoleWriter .NOTES General notes #> [CmdletBinding()] param() if
($PSVersionTable.PSVersion.Major -gt 3) { return } try { #
http://www.leeholmes.com/blog/2008/07/30/workaround-the-os-handles-position-is-not-
what-filestream-expected/ plus comments $bindingFlags = [Reflection.BindingFlags]
"Instance,NonPublic,GetField" $objectRef = $host.GetType().GetField("externalHostRef",
$bindingFlags).GetValue($host) $bindingFlags = [Reflection.BindingFlags]
"Instance,NonPublic,GetProperty" $consoleHost =
$objectRef.GetType().GetProperty("Value", $bindingFlags).GetValue($objectRef, @()) [void]
$consoleHost.GetType().GetProperty("IsStandardOutputRedirected",
$bindingFlags).GetValue($consoleHost, @()) $bindingFlags = [Reflection.BindingFlags]

```

```

"Instance,NonPublic,GetField" $field =
$consoleHost.GetType().GetField("standardOutputWriter", $bindingFlags)
$field.SetValue($consoleHost, [Console]::Out) [void]
$consoleHost.GetType().GetProperty("IsStandardErrorRedirected",
$bindingFlags).GetValue($consoleHost, @()) $field2 =
$consoleHost.GetType().GetField("standardErrorWriter", $bindingFlags)
$field2.SetValue($consoleHost, [Console]::Error) } catch { Write-Warning "Unable to apply
redirection fix." } } function Test-ChocolateyInstalled { [CmdletBinding()] param()
$checkPath = if ($env:ChocolateyInstall) { $env:ChocolateyInstall } else {
"$env:PROGRAMDATA\chocolatey" } if ($Command = Get-Command choco -CommandType
Application -ErrorAction Ignore) { # choco is on the PATH, assume it's installed Write-
Warning "'choco' was found at '$($Command.Path)'." $true } elseif (-not (Test-Path
$checkPath)) { # Install folder doesn't exist $false } else { # Install folder exists if (Get-
ChildItem -Path $checkPath) { Write-Warning "Files from a previous installation of
Chocolatey were found at '$($CheckPath)'." } # Return true here to prevent overwriting an
existing installation $true } } function Install-7zip { [CmdletBinding()] param(
[Parameter(Mandatory = $true)] [string] $Path, [Parameter(Mandatory = $false)]
[hashtable] $ProxyConfiguration ) if (-not (Test-Path ($Path))) { Write-Host "Downloading 7-
Zip commandline tool prior to extraction." Request-File -Url
'https://community.chocolatey.org/7za.exe' -File $Path -ProxyConfiguration
$ProxyConfiguration } else { Write-Host "7zip already present, skipping installation." } }
#endregion Functions #region Pre-check # Ensure we have all our streams setup correctly,
needed for older PSVersions. Set-PSConsoleWriter if (Test-ChocolateyInstalled) { $message
= @( "An existing Chocolatey installation was detected. Installation will not continue. This
script will not overwrite existing installations." "If there is no Chocolatey installation at
'$env:ChocolateyInstall', delete the folder and attempt the installation again." "" "Please
use choco upgrade chocolatey to handle upgrades of Chocolatey itself." "If the existing
installation is not functional or a prior installation did not complete, follow these steps:" " -
Backup the files at the path listed above so you can restore your previous installation if
needed." " - Remove the existing installation manually." " - Rerun this installation script." "
- Reinstall any packages previously installed, if needed (refer to the lib folder in the
backup)." "" "Once installation is completed, the backup folder is no longer needed and
can be deleted." ) -join [Environment]::NewLine Write-Warning $message return }
#endregion Pre-check #region Setup $proxyConfig = if ($IgnoreProxy -or -not $ProxyUrl) {
@{} } else { $config = @{ ProxyUrl = $ProxyUrl } if ($ProxyCredential) {
$config['ProxyCredential'] = $ProxyCredential } elseif ($env:chocolateyProxyUser -and
$env:chocolateyProxyPassword) { $securePass = ConvertTo-SecureString
$env:chocolateyProxyPassword -AsPlainText -Force $config['ProxyCredential'] =
[System.Management.Automation.PSCredential]::new($env:chocolateyProxyUser,
$securePass) } $config } # Attempt to set highest encryption available for
SecurityProtocol. # PowerShell will not set this by default (until maybe .NET 4.6.x). This #
will typically produce a message for PowerShell v2 (just an info # message though) try { #
Set TLS 1.2 (3072) as that is the minimum required by Chocolatey.org. # Use integers
because the enumeration value for TLS 1.2 won't exist # in .NET 4.0, even though they are

```

addressable if .NET 4.5+ is # installed (.NET 4.5 is an in-place upgrade). Write-Host "Forcing web requests to allow TLS v1.2 (Required for requests to Chocolatey.org)"

```
[System.Net.ServicePointManager]::SecurityProtocol =
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072 } catch { $errorMessage =
@( 'Unable to set PowerShell to use TLS 1.2. This is required for contacting Chocolatey as
of 03 FEB 2020.' 'https://blog.chocolatey.org/2020/01/remove-support-for-old-tls-versions/'
'If you see underlying connection closed or trust errors, you may need to do one or more of
the following:' '(1) upgrade to .NET Framework 4.5+ and PowerShell v3+,' '(2) Call
[System.Net.ServicePointManager]::SecurityProtocol = 3072; in PowerShell prior to
attempting installation,' '(3) specify internal Chocolatey package location (set
$env:chocolateyDownloadUrl prior to install or host the package internally),' '(4) use the
Download + PowerShell method of install.' 'See https://docs.chocolatey.org/en-
us/choco/setup for all install options.' ) -join [Environment]::NewLine Write-Warning
$errorMessage } if ($ChocolateyDownloadUrl) { if ($ChocolateyVersion) { Write-Warning
"Ignoring -ChocolateyVersion parameter ($ChocolateyVersion) because -
ChocolateyDownloadUrl is set." } Write-Host "Downloading Chocolatey from:
$ChocolateyDownloadUrl" } elseif ($ChocolateyVersion) { Write-Host "Downloading
specific version of Chocolatey: $ChocolateyVersion" $ChocolateyDownloadUrl =
"https://community.chocolatey.org/api/v2/package/chocolatey/$ChocolateyVersion" } else {
Write-Host "Getting latest version of the Chocolatey package for download." $queryString
= [uri]::EscapeUriString("(Id eq 'chocolatey') and (not IsPrerelease)) and IsLatestVersion")
$queryUrl = 'https://community.chocolatey.org/api/v2/Packages()?$filter={0}' -f
$queryString [xml]$result = Request-String -Url $queryUrl -ProxyConfiguration
$proxyConfig $ChocolateyDownloadUrl = $result.feed.entry.content.src } if (-not
$env:TEMP) { $env:TEMP = Join-Path $env:SystemDrive -ChildPath 'temp' } $chocoTempDir
= Join-Path $env:TEMP -ChildPath "chocolatey" $tempDir = Join-Path $chocoTempDir -
ChildPath "chocoInstall" if (-not (Test-Path $tempDir -PathType Container)) { $null = New-
Item -Path $tempDir -ItemType Directory } #endregion Setup #region Download & Extract
Chocolatey $file = Join-Path $tempDir "chocolatey.zip" # If we are passed a valid local
path, we do not need to download it. if (Test-Path $ChocolateyDownloadUrl) { Write-Host
"Using Chocolatey from $ChocolateyDownloadUrl." Copy-Item -Path
$ChocolateyDownloadUrl -Destination $file } else { Write-Host "Getting Chocolatey from
$ChocolateyDownloadUrl." Request-File -Url $ChocolateyDownloadUrl -File $file -
ProxyConfiguration $proxyConfig } Write-Host "Extracting $file to $tempDir" if
($PSVersionTable.PSVersion.Major -lt 5) { # Determine unzipping method # 7zip is the
most compatible pre-PSv5.1 so use it unless asked to use builtin if ($UseNativeUnzip) {
Write-Host 'Using built-in compression to unzip' try { $shellApplication = New-Object -
ComObject Shell.Application $zipPackage = $shellApplication.Namespace($file)
$destinationFolder = $shellApplication.Namespace($tempDir)
$destinationFolder.CopyHere($zipPackage.Items(), 0x10) } catch { Write-Warning "Unable
to unzip package using built-in compression. Set $env:chocolateyUseWindowsCompression =
'' or omit -UseNativeUnzip and retry to use 7zip to unzip." throw $_ } } else { $7zaExe =
Join-Path $tempDir -ChildPath '7za.exe' Install-7zip -Path $7zaExe -ProxyConfiguration
$proxyConfig $params = 'x -o"{0}" -bd -y "{1}"' -f $tempDir, $file # use more robust
```

```
Process as compared to Start-Process -Wait (which doesn't # wait for the process to finish
in PowerShell v3) $process = New-Object System.Diagnostics.Process try {
$process.StartInfo = New-Object System.Diagnostics.ProcessStartInfo -ArgumentList
$7zaExe, $params $process.StartInfo.RedirectStandardOutput = $true
$process.StartInfo.UseShellExecute = $false $process.StartInfo.WindowStyle =
[System.Diagnostics.ProcessWindowStyle]::Hidden $null = $process.Start()
$process.BeginOutputReadLine() $process.WaitForExit() $exitCode = $process.ExitCode }
finally { $process.Dispose() } $errorMessage = "Unable to unzip package using 7zip.
Perhaps try setting `$_env:chocolateyUseWindowsCompression = 'true' and call install
again. Error:" if ($exitCode -ne 0) { $errorDetails = switch ($exitCode) { 1 { "Some files
could not be extracted" } 2 { "7-Zip encountered a fatal error while extracting the files" }
7 { "7-Zip command line error" } 8 { "7-Zip out of memory" } 255 { "Extraction cancelled
by the user" } default { "7-Zip signalled an unknown error (code $exitCode)" } } throw
($errorMessage, $errorDetails -join [Environment]::NewLine) } } else {
Microsoft.PowerShell.Archive\Expand-Archive -Path $file -DestinationPath $tempDir -Force }
#endregion Download & Extract Chocolatey #region Install Chocolatey Write-Host
"Installing Chocolatey on the local machine" $toolsFolder = Join-Path $tempDir -ChildPath
"tools" $chocoInstallPS1 = Join-Path $toolsFolder -ChildPath "chocolateyinstall.ps1" &
$chocoInstallPS1 Write-Host 'Ensuring Chocolatey commands are on the path'
$chocoInstallVariableName = "ChocolateyInstall" $chocoPath =
[Environment]::GetEnvironmentVariable($chocoInstallVariableName) if (-not $chocoPath) {
$chocoPath = "$_env:ALLUSERSPROFILE\Chocolatey" } if (-not (Test-Path ($chocoPath))) {
$chocoPath = "$_env:PROGRAMDATA\chocolatey" } $chocoExePath = Join-Path $chocoPath -
ChildPath 'bin' # Update current process PATH environment variable if it needs updating. if
($_env:Path -notlike "$chocoExePath") { $_env:Path =
[Environment]::GetEnvironmentVariable('Path',
[System.EnvironmentVariableTarget]::Machine); } Write-Host 'Ensuring chocolatey.nupkg is
in the lib folder' $chocoPkgDir = Join-Path $chocoPath -ChildPath 'lib\chocolatey' $nupkg =
Join-Path $chocoPkgDir -ChildPath 'chocolatey.nupkg' if (-not (Test-Path $chocoPkgDir -
PathType Container)) { $null = New-Item -ItemType Directory -Path $chocoPkgDir } Copy-
Item -Path $file -Destination $nupkg -Force -ErrorAction SilentlyContinue #endregion Install
Chocolatey # SIG # Begin signature block #
MIInKwYJKoZIhvcNAQcCollnHDCCJxgCAQExDzANBgIghkgBZQMEEAgEFADB5Bgor #
BgEEAYI3AgEEOGswATA0BgorBgEEAYI3AgEeMCYCAwEAAAQQH8w7YFILCE63JNLG #
KX7zuUIBAAIBAAIBAAIBADAxMA0GCWCGSAFIawQCAQUABCDh6HLVOQJjm0Ac #
djgglgACeRd7TPrNslkAKPOISM1vq6CCIK4wggWNMIIEdaADAgECAhAOMxiO+dAt #
5+/bUOIIBhaMA0GCSqGSib3DQEBDAUAMGUxCzAJBgNVBAYTAIVTMRUwEwYDVQQK #
EwxEaWdpQ2VydCBjbMmGTAXBgNVBASTEhd3dy5kaWdpY2VydC5jb20xJDAlBgNV #
BAMTG0RpZ2IDZXJ0IEFzc3VyZWQgSUQgUm9vdCBDQTAEfw0yMjA4MDAwMDBa #
Fw0zMTEyMTExMDkyMzU5NTlaMGlxGzAJBgNVBAYTAIVTMRUwEwYDVQQKEwxEaWdpQ2VydCBjbMmGTAXBgNVBASTEhd3dy5kaWdpY2VydC5jb20xITAfBgNVBAMTG0RpZ2IDZXJ0IFRydXN0ZWQgUm9vdCBHNDCCAilwDQYJKoZIhvcNAQEBAQADggIPADCCAgOC #
ggIBAL/mkHN03rvkXUo8MCIwaTPswqcILskhPfKK2FnC4SsmNPVirdprNrnsbhA3E #
MB/zG6Q4FutWxpdtHauyefLKEdLkX9YFPFPUh/GnhWlfr6fqVcWWVVyr2iTcMKy #
```

unWZanMyIINEQRBAu34LzB4TmdDttceltDBvuINXJIB1jKS3O7F5OyJP4IWGbNOsF #
xl7sWxq868nPzaw0QF+xembud8hlqGZXV59UWI4MK7dPpzDZVu7Ke13jrcIPXuU1 #
5zHL2pNe3I6PgNq2kZhAkHnDeMe2scS1ahg4AxCN2NQ3pC4FfYj1gj4QkXCrvYJB #
MtfbBHMqbpEBfCFM1LyuGwN1XXhm2ToxRJozQL8l11pJpMLmqaBn3aQnvKFPObUR #
WBf3JfXGj2T3wWmldph2PVldQnaHiZdpekjw4KISG2aadMreSx7nDmOu5tTvKpl6 #
nj3cAORFJYm2mkQZK37AILTSYW3rM9nF30sEAMx9HJXDj/chsrIRt7t/8tWMcCxB #
YKqxYxhElRp2Yn72gLD76GSmM9GJB+G9t+ZDpBi4pncB4Q+UDCEdslQpJYls5Q5S #
UUd0viastkF13nqsX40/ybzTQRESW+UQUOssxcpyFilJ33xMdT9j7CFfxCBRa2+x #
q4aLT8LWRV+dIPyhHsXAJ6KxfgommfXkaS+YHS312amyHeUbAgMBAAGjggE6MIIB #
NjAPBgNVHRMBAf8EBTADAQH/MB0GA1UdDgQWBBS1+OC0nFdZEzflmc/57qYrhwp #
TzAfBgNVHSMEGDAWgBRF66Kv9JLLGjEtUYunpyGd823IDzAOBgNVHQ8BAf8EBAMC #
AYYweQYIKwYBBQUHAQEEbTBrMCQGCGCCsGAQUFBzABhhhodHRwOi8vb2NzcC5kaWdp #
Y2VydC5jb20wQwYIKwYBBQUHMAKGN2h0dHA6Ly9jYWNLcnRzLmRpZ2ZlZXJ0LmNv #
bS9EaWdpQ2VydEFzc3VyZWVjZjB3RDQ55jcnQwRQYDVR0fBD4wPDA6oDigNoY0 #
aHR0cDovL2NybdMuZGlnaWNlcnQuY29tL0RpZ2ZlZXJ0QXNzdXJlZEIEUm9vdENB #
LmNybdARBgNVHSAECjAIMAYGBFUDIAAwDQYJKoZIhvcNAQEMBQADggEBAHCgv0Nc #
Vec4X6CjdBs9thbX979XB72arKGHLOyFXqkauyL4hxppVCLtpIh3bb0aFPQTSnov #
Lbc47/T/gLn4offyct4kvFIDyE7QKt76LVbP+ft3rDB6mouyXtTP0UNEm0Mh65Zy #
oUi0mcudT6cGAxN3J0TU53/oWajwvy8LpunyNDzs9wPHh6jSTEAZNUZqaVSwuKFW #
juyk1T3osdz9HNj0d1pcVlxv76FQPfx2CWiEn2/K2yCNNWAcAgPLILCsWKAQQGPF #
mCLBslN1VWvPJ6tsds5vly30fnFql2si/xK4VC0nftg62fC2h5b9W9FcrBjDTZ9z #
twGpn1eqXijiuZQwggauMIIElqADAgECAhAHNje3JFR82Ees/ShmKI5bMA0GCSqG #
S1b3DQEBcWUAMGlxCzAJBgNVBAYTAIVTMRUwEwYDVQQKEwxEaWdpQ2VydCBJbmMx #
GTAXBgNVBAsTEHd3dy5kaWdpY2VydC5jb20xITAFBgNVBAMTGERpZ2ZlZXJ0IFRy #
dXN0ZWQgUm9vdCBHNDAAeFw0yMjAzMjMwMDAwMDBaFw0zNzAzMjlyMzU5NTlaMGMx #
CzAJBgNVBAYTAIVTMRcwFQYDVQQKEw5EaWdpQ2VydCwgSW5jLjE7MDkGA1UEAxMy #
RGlnaUNlcnQgVHJ1c3RlZCBHNCBSU0E0MDk2IFNIQTl1NiBUaW1lU3RhbXBpbmcg #
Q0EwgglMA0GCSqGS1b3DQEBAQUAA4ICDwAwggIKAoICAQDGhjUGSbPBPXJJUVXH #
JQPE8pE3qZdRodbSg9GeTKJtoLDMg/la9hGhRBVCX6SI82j6ffOciQt/nR+eDzMf #
UBMLJnOWbfhXqAJ9/U00hNoR8XOxs+4rglSKlhjf69o9xBd/qxkrPkLcZ47qUT3w #
1lbU5ygt69OxtXXnHwZlJZQp09nsad/ZkldGAHvbREGJ3HxqV3rwN3mfXazL6lRk #
tFLydkf3YYMZ3V+0VAshaG43lbtArF+y3kp9zvU5EmfvDqVjbOSmxR3NNg1c1eYb #
qMFkdECnwHLFuk4fsbVYTXn+149zk6wsOeKISNbwsDETqVcplicu9Yemj052FVUm #
cjgmf6AaRyBD40NjgHt1biclkJg6OBGz9vae5jtb7IHelhTZgirHkr+g3uM+onP6 #
5x9abJTyUpURK1h0QCirc0PO30qhHG54xSnzyqqWc0Jon7ZGs506o9UD4L/wojzK #
QtwYSH8UNM/STKvvmz3+DrhkKvp1KCRB7UK/BZxmSVJQ9FHzNklNiyDSLfc1eSuo #
80VgvCONWPfcYd6T/jnA+blwpUzX6ZhKWD7TA4j+s4/TXkt2ElGTyYwMO1uKlqjB #
Jgj5FBASA31fi7tk42PgpuE+9sj0sj8eCXbsq11GdeJgo1gJASgADoRU7s7pXche #
MBK9Rp6103a50g5rmQzSM7TnsQIDAQABo4IBXTCCAkwEgYDVR0TAQH/BAgwBgEB #
/wIBADAdBgNVHQ4EFgQUuhbZbU2FL3MpdpovdYxqll+eyG8wHwYDVR0jBBgwFoAU #
7NfjgtjXWRM3y5nP+e6mK4cD08wDgYDVR0PAQH/BAQDAgGGMBMGA1UdJQQMMAoG #
CCsGAQUFBwMIMHcGCCsGAQUFBwEBBGswaTAKBggrBgEFBQcwAYYYaHR0cDovL29j #
c3AuZGlnaWNlcnQuY29tMEEGCCsGAQUFBzACHjVodHRwOi8vY2FjZXJ0cy5kaWdp #
Y2VydC5jb20vRGlnaUNlcnRUcnVzdGVkUm9vdEc0LmNydDBDBGNVHR8EPDA6MDig #

NqA0hjJodHRwOi8vY3JsMy5kaWdpY2VydC5jb20vRGlnaUNlcnRUcnVzdGVkUm9v #
dEc0LmNyBDAgBgNVHSAEGTAXMAgGBmeBDAEEAjALBgIghkgBhv1sBwEwDQYJKoZI #
hvcNAQELBQADggIBAH1ZjsCTtm+YqUQiAX5m1tghQuGwGC4QTRPPMFPOvxj7x1Bd #
4ksp+3CKDaopafxpwc8dB+k+YMjYC+VcW9dth/qEICU0MWfNthKWb8RQTGIldDAiC #
qBa9qVbPFXONASIlzpVpP0d3+3J0FNf/q0+KLHqrhc1DX+1gtqpPkWaeLJ7giqzl #
/Yy8ZCaHbJK9nXzQcAp876i8dU+6WvepELjd6f8oVlnw1YpxdmXazPByoyP6wCeC #
RK6ZJxurJB4mwbfeKuv2nrF5mYGjVoarCkXJ38SNoOeY+/umnXKvxMfBwWpx2cYT #
gAnEtp/Nh4cku0+jSbl3ZpHxczpSjSpzd+k1OsOx0ISQ+UzTI63f8IY5knLD0/ #
a6fxZsNBzU+2QJshlUDQtxMkzdwdeDrknq3INHGS1yZr5Dhzq6YBT70/O3itTK37 #
xjV77QpfMzmHQXh6Oomc4d0j/R0o08f56PGYX/sr2H7yRp11LB4nLCbbbxV7HhmL #
NriT1ObyF5lZynDwN7+YAN8gFk8n+2BnFqFmut1VwDophrCYoCvtlUG3OtUVmDG0 #
YgkPCr2B2RP+v6TR81fZvAT6gt4y3wSJ8ADNXcL50CN/AAvkdglm2fBldkKmKYcj #
RyvmfxqkhQ/8mjb2VVQRH4D6wPIOK+XW+6kVRBVK5xMOHds3OBqhK/bt1nz8MIIG #
sDCCBJigAwIBAgIQCK1AsmDSnEyfXs2pvZOu2TANBgkqhkiG9w0BAQwFADBiMQsw #
CQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3d3cu #
ZGlnaWNlcnQuY29tMSEwHwYDVQQDEhEaWdpQ2VydCBUcnVzdGVkIFJvb3QgRzQw #
HhcNMjEwNDI5MDAwMDAwWhcNMzYwNDI4MjM1OTU5WjBpMQswCQYDVQQGEwJVUzEX #
MBUGA1UEChMORGlnaUNlcnQsIEluYy4xQTA/BgNVBAMTOERpZ2IDZXJ0IFRydXN0 #
ZWQgRzQgQ29kZSBTaWduaW5nIFJTQTQwOTYgU0hBMzg0IDlwMjEgQ0ExMlIjAN #
BgkqhkiG9w0BAQEFAAOCAg8AMIICGgKCAgEA1bQvQtAorXi3XdU5WRuxiEL1M4zr #
PYGXcMW7xIUmmJ+kjmyXPXrNCQH4UtP03hD9BfXhtr50tVnGljPDqFX/liZwZHM #
gQM+TXAkZLON4gh9NH1MgFcSa0OamfLFOx/y78tHWhOmTLMBICXzENOLsvsl8lrg #
nQnAZaf6mlBJNYc9URNokCF4RS6hnyzhGmlazMXuk0lwQjKP+8bqHPNlaJGiTuYc #
EUhSaN4QvRRXXegYE2Xff7JPhSxlpFaENdb5LpyqABXRN/4aBpTCfMjqGzLmysL0 #
p6MDDnSlrzm2q2AS4+jWufcx4dyt5Big2MEjR0ezoQ9uo6ttmAaDG7dqZy3SvUQa #
khCBj7A7CdfHmzJawv9qYFSLScGT7eG0XOBv6yb5jNWY+TgQ5urOkfW+/0/tvk2E0 #
XLyTRSiDnlpKF+wc86LjiUGsoPUXPYVGUztYuBeM/Lo6OwKp7ADK5GyNnm+960I #
HnWmZcy740hQ83eRGv7bUKJGyGFYmPV8AhY8gyitOYbs1LcNU9D4R+Z1MI3sMJN2 #
FKZbS110YU0/EpF23r9Yy3IQKUHw1cVtJnZoEUETWJrcjisB9IINWdt4z4FKPkBH #
X8mBUHOFECMhWWCKZFTBzCEa6DgZfGYczXg4RTCZT/9jT0y7qg0IU0F8WD1Hs/q2 #
7lwyCQLMbDwMVhECAwEAAaOCAVkwggFVMBIGA1UdEwEB/wQIMAYBAf8CAQAwHQYD #
VR0OBBYEFgG34Ou2O/hfEYb7/mF7CIhI9E5CMB8GA1UdIwQYMBaAFOzX44LScV1k #
TN8uZz/nupiuHA9PMA4GA1UdDwEB/wQEAwIBhjATBgNVHSUEDDAKBggrBgEFBQcD #
AzB3BggrBgEFBQcBAQRrMGkwJAYIKwYBBQUHMAGGGGh0dHA6Ly9vY3NwLmRpZ2lj #
ZXJ0LmNvbTB3BggrBgEFBQcwAoY1aHR0cDovL2NhY2VydHMuZGlnaWNlcnQuY29t #
L0RpZ2IDZXJ0VHJ1c3RIZFJvb3RHNC5jcnQwQwYDVROfBDwwOjA4oDagNIYyaHR0 #
cDovL2NybdMuZGlnaWNlcnQuY29tL0RpZ2IDZXJ0VHJ1c3RIZFJvb3RHNC5jcmww #
HAYDVR0gBBUwEzAHBgVngQwBAzAIBgZngQwBBAEwDQYJKoZIhvcNAQEMBQADggIB #
ADojRD2NCHbuj7w6mdNW4AlapfhINPMstuZ0ZveUcrEAyq9sMCcTEp6QRJ9L/Z6j #
fCbVN7w6XUhtldU/SfQnuxaBRVD9nL22heB2fjdxxyL3WqqQz/WTauPrINHvUHml #
moqKwba9oUgYftzYgBoRGRjNYZmBVvbJ43bnxOQbX0P4PpT/djk9ntSZz0rdKOtf #
JqGVWEjVGv7Xjz/9kNF2ht0csGBc8w2o7uCJob054ThO2m67Np375SFTWsPK6Wrx #
oj7bQ7gzyE84FJKZ9d3OVG3ZXQIUH0AzaFpIlbLCIXVzUstG2MQ0HKKIS43Nb3Y3 #
LIU/Gs4m6Ri+kAewQ3+ViCCcCPDMyu/9KTVch4k4Vfc3iosJocsL6TEa/y4ZXDlx #

4b6cpwoG1iZnt5LmTl/eeqxJzy6kdJKt2zyknlYf48FWGysj/4+16oh7cGvmolr9 #
Oj9FpsToFpFSi0HASIRLlk2rREDjffAVKM7t8RhWByovEMQMCGQ8M4+uKlw8y4+I #
Cw2/O/TOHnuO77Xry7fwdxPm5yg/rBKupS8ibEH5glwVZsxsDsrFhsP2JjMMB0ug #
0wcCampAMEhLNKhRlLutG4UI4IkNbcoFUCvqShyepf2gpx8GdOfy1IKQ/a+FSCH5 #
Vzu0nAPthkX0tGFuv2jijmCG6sivqf6UHedjGzqGVnhOMIIGwjCCBKqgAwlBAglQ #
BUSv85SdCDmmv9s/X+VhFjANBgkqhkiG9w0BAQsFADBJMQswCQYDVQQGEWJlUzEX #
MBUGA1UEChMORGlnaUNlcnQsIEluYy4xOzA5BgNVBAMTMkRpZ2IDZXJ0IFRydXN0 #
ZWQgRzQgUINBNDAA5NiBTSEEEyNTYgVGltZVN0YW1waW5nIENBMB4XDTIzMDcxNDAw #
MDAwMFoXDTE0MTAxMzIzNTk1OVowSDELMakGA1UEBhMCVVMxZmFzAVBgNVBAoTDkRp #
Z2IDZXJ0LCBjb2MMSAwHgYDVQQDEXdEaWdpQ2VydCBUaW1lc3RhbXAAMjAyMzCC #
AilwDQYJKoZIhvcNAQEBBQADggIPADCCAgoCggIBAKNTRYcdg45brD5UsyPgZ5/X #
5dLnXaEOCdwwSKOXejsqnGfcYhVYwamTEafNqrJq3RApih5iY2nTWJw1cb86l+uU #
UI8clOrHmjsvlmbjaedp/lvD1isgHMGXILSIUIHyz8sHpjBoyoNC2vx/CSSUpIla #
2mq62DvKXd4ZGIX7ReoNYWyd/nFexAaaPPDFLnkPG2ZS48jWPI/aQ9OE9dDH9kgt #
Xkv1lnX+3RChG4PBuOZSlbVH13gpOWvgeFmX40QrStWVzu8IF+qCZE3/I+PKhu60 #
pCFkcOvV5aDaY7Mu6QXuqvYk9R28mxyyt1/f8O52fTGZZUdVnUokL6wrl76f5P17 #
cz4y7II0+9S769SgLDSb495uZBkHNwGRDxy1Uc2qTgaDiGhiu7xBG3gZbeTZD+BY #
QfvYsSzhUa+0rRUGFOpiCBPTaR58ZE2dD9/O0V6MqqtQFcmzyrzXxDtoRKOIO0L9 #
c33u3Qr/eTQQfqZcClhMAD6FaXXHg2TWdc2PEnzWpST618RrlbroHzSYLzrqawGw #
9/sqhux7UjipmAmhcbjsca8+uG+W1eEQE/5hRwqM/vC2x9XH3mwk8L9CgsqgcT2c #
kpMEtGlWjw1Pt7U20clfCKRwo+wK8REuZODLlivK8SgTIUIRfgZm0zu++uuRONhR #
B8qUt+JQofM604qDy0B7AgMBAAGjggGLMIIBhZAQBgNVHQ8BAf8EBAMCB4AwDAYD #
VR0TAQH/BAIwADAwBgNVHSUBAf8EDDAKBggrBgEFBQcDCDAgBgNVHSAEGTAXMAGG #
BmeBDAAEEAJALBglghkgBhvh1sBwEwHwYDVR0jBBgwFoAUuhbZbU2FL3MpdpovdYxq #
II+eyG8wHQYDVR0OBBYEFKW27xPn783QZKHVVqllMaPe1eNjMFOGA1UdHwRTMFew #
T6BNoEuGSWh0dHA6Ly9jcmwzLmRpZ2ljZXJ0LmNvbS9EaWdpQ2VydFRydXN0ZWRH #
NFJTQTQwOTZTSEEEyNTZUaW1U3RhbXBpbmdDQS5jcmwwgZAGCCsGAQUFBwEBBIGD #
MIGAMCQGCCsGAQUFBzABhhhodHRwOi8vb2Nzc5kaWdpY2VydC5jb20wWAYIKwYB #
BQUHMAKGTGh0dHA6Ly9jYWNlcnRzLmRpZ2ljZXJ0LmNvbS9EaWdpQ2VydFRydXN0 #
ZWRHNFJTQTQwOTZTSEEEyNTZUaW1U3RhbXBpbmdDQS5jcnQwDQYJKoZIhvcNAQEL #
BQADggIBAEa1t6gqbWYF7xwJU+KPGic2CX/yyzkzepdlpLsjCICbjPgKjZ5+PF #
7SaCinEvGN10tt5s1+FgnCvt7T1ljhrunxdvcJhN2hJd6PrkKoS1yeF844ektrC #
QDifXcigLiV4JZ0qBXqEKZi2V3mP2yZWK7Dzp703DNiYdk9WuVLCTp04qYHnbUFc #
jGnRuSvExnvPnPp44pMadqJpddNQ5EQSviANnqlE0PjISXciWiHfTm+YIRpUurm8 #
wWkZus8W8oM3NG6wQSbd3lqXTzON1I13fXVFOaVYJmoDRd7ZULVQJk9WvUzF4UbF #
KN0t50MAcN7Mmj4ZiQPq1JE3701S88lglcRWR+3aEUuMMsOI5ljitts++V+wQtaP #
4xeR0arAVeOGv6wnLEHQmjNKqDbUuXKWfPd5OEhfysLcPTLfddY2Z1qJ+Panx+VP #
NTwAvb6cKmx5AdzaROY63jg7B145WPR8czFVoIARyxQMfq68/qTreWWqaNYiyjvr #
mol1VygWy2nyMpqy0tg6uLFGhmu6F/3Ed2wVbK6rr3M66ElGt9V/zLY4wNjsHPW2 #
obhDLN90TH0eaHDAdwrUAuBcYLso/zjlUlRWrBcil0707NMx+1Br/wd3H3GXREHJ #
uEbTbDJ8WC9nR2XIG3O2mflrLAZG70Ee8PBf4NvZrZCARK+AEEGKMIIG7TCCBNWg #
AwlBAglQBNI793fHTneCMtwLiiYFTANBgkqhkiG9w0BAQsFADBpMQswCQYDVQQG #
EWJlUzEXMBUGA1UEChMORGlnaUNlcnQsIEluYy4xQTA/BgNVBAMTOERpZ2IDZXJ0 #
IFRydXN0ZWQgRzQgQ29kZSBTaWduaW5nIFJTQTQwOTYgU0hBMzq0IDlwMjEgQ0Ex #

MB4XDTI0MDUwOTAwMDAwMFoXDTI3MDUxMTIzNTk1OVowdTELMaKGA1UEBhMCMVVMx #
DzANBgNVBAgTBkthbnNhcZEPMA0GA1UEBxMGVVG9wZWthMSEwHwYDVQQKEzhDaG9j #
b2xhdGV5IFNvZnR3YXJLCBjbmMxITAFBgNVBAMTGENob2NvbGF0ZXkgU29mdHdh #
cmUslEluYzCCAalwDQYJKoZIhvcNAQEBBQADggGPADCCAYoCggGBAPDJgdZWj0RV #
lBBBniCyGy19FB736U5AahB+dAw3nmafoEEg+syql0m9kzV0gu4bSd4AI587ioAG #
DUPAGhXf0R+y11cx7c1cgdyxvfBvfMEkgD7sOUeF9ggZjc0YZ4qc7Pa6qqMpHDru #
pjshvLmQMSLaGKF68m+w2mjiZkLMYBEotPiAC3+lzl1MQqidCfN6rfQUmtcKyrVz #
2zCt8CvuR3pSyNCBcQgKZ/+NwBfDqPTt1wKq5JCIQiLnbDZwj9F5433enzgUGQgh #
KRolwfp/hap7t7lrNf859Xe1/zHT4qtNgzGqSdJ2Kbz1YAMFjZokYHv/sliyxJN9 #
7++0BApX2t45JsQaqyQ60TSKxqOH0JIIDeYgwxjf8YFmuv7T4zVM8u02Axp/1YV #
nKP2AOVca6FDe9EiccrexAWPGOP+WQi8WFQKrNVKr5XTLI0MNTjadOHff0XUToyF #
H8FVnZZV1/F1kgd/bYbt/0M/QkS4FGmJqT8dyRyMkTITynKul4N3QIDAQABo4IC #
AzCCAF8wHwYDVR0jBBGwFoAUaDfg67Y7+F8Rhvv+YXsliGX0TklwHQYDVR0OBBYE #
FFpfZUilS5A+fjYV80ib5qKkBoczMD4GA1UdIAQ3MDUwMwYGGZ4EMAQQBMCkwjwYI #
KwYBBQUHAgEWG2h0dHA6Ly93d3cuZGlnaWNlcnQuY29tL0NQZzA0BgNVHQ8BAf8E #
BAMCB4AwEwYDVR0IBAwWcGyIKwYBBQUHAWMwgbUGA1UdHwSBrTCBqjBToFGgT4ZN #
aHR0cDovL2NybdMuZGlnaWNlcnQuY29tL0RpZ2IDZXJ0VHJ1c3RIZEc0Q29kZVNp #
Z25pbmdSU0E0MDk2U0hBMzg0MjAyMUNBM55jcmwwU6BRoE+GTWh0dHA6Ly9jcmw0 #
LmRpZ2ljZXJ0LmNvbS9EaWdpQ2VydFRydXN0ZWRRHNENvZGVtWduW5nUINBNDAA5 #
NINIQTm4NDIwMjFDQTEuY3JsMIGUBggrBgEFBQcBAQSBhzCBhDAKBggrBgEFBQcw #
AYYYaHR0cDovL29jc3AuZGlnaWNlcnQuY29tMFwGCCsGAQUFBzACHlBodHRwOi8v #
Y2FjZXJ0cy5kaWdpY2VydC5jb20vRGlnaUNlcnRUcnVzdGVkRzRDb2RIU2lnbmlu #
Z1JlTQTQwOTZTSEezODQyMDIxQ0ExLmNydDAJBgNVHRMEAjAAMA0GCSqGSIb3DQEB #
CwUAA4ICAQAW9ANNkr2cF6ulbM+/XUWeWqC7UTqtsRwj7WAo8XTr52JebRchTGDH #
BZP9sDRZsFt+IPcPvBrv41kWoAFBmebTaPMh6YDHaON+uc19CTWXsMh8eog0lzGU #
iA3mKdbVit0udrgNIBUqTlUvMIMFIARWSz90FMeQrCFokLmqoqjp7u0sVPM7ng6T #
9D8ct/m5LSpla5TJCjAfyfw75GK0wzTDdT1MgiAlyX0EedMrEwXjOjSAPQ+uhIW #
v/AHDf8ukJzDFTTeiUkYZ1w++z70QZkzLfQTI6eH9vqgyXWcnGCwOxKquqe8RSle #
M3FdtLstn9nl8S4qeiKdmomG6FAZTzYiGULjdJGsLh6Uii56zZdq3bSre/yrfed4 #
hf/0MqEtWSU7LpkWM8AApRkIKRBZIQ73/7WxwsF9kHoZxqoRMDGTzWt+S7/XrSOa #
QbKf0CxdxMPHKC2A1u3xGNDChTQEwpHxYXf/teD7GeFYFQJg/wn4dC72mZze97+c #
Ycpml4R13Q7owmRthK1hnuq4EOQlcoTPbQXiaRzULbYrcOnji7EbXcqdeAAAnZAYV #
b6zGqAaE9Sw4RYvkosL5IIBgrdlwSFJMbeirBoM2GukIHQ8UaEu3l1PoNQvVbqM1 #
8zHiN4WA4rp9G9wfcAlZWq9iKF34sA+Xu03qSVaKPKn6YJMI5PfUsDGCBdMwggXP #
AgEBMH0waTELMaKGA1UEBhMCMVVMxZzAVBgNVBAoTDkRpZ2IDZXJ0LCBjbmMuMUEw #
PwYDVQQDEzhEaWdpQ2VydCBUcnVzdGVkIEc0IENvZGUgU2lnbmluZyBSU0E0MDk2 #
IFNIQTm4NCAyMDIxIENBMQIQBNi793fIHtneCMtwLiiYFTANBgIghkgBZQMEAgEF #
AKCBhDAYBgorBgEEAYI3AgEMMQowCKACGACHAoAAMBKGCsGSIb3DQEJAzEMBgor #
BgEEAYI3AgEEMBWGCisGAQQBgjcCAQsxDjAMBgorBgEEAYI3AgEVMC8GCsGSIb3 #
DQEJBDEiBCBVpza7GC5PEQSeEyh6E4PiLRHX8M57kjBOS3cO32zazDANBgkqhkiG #
9w0BAQEFAASCAyDHikMKs7NgnG6w7RkEy1HCA74YrZ/nF4twOoy+i3k/2Mm0LvKC #
BO3y+L5Wjg2WfsoAFQwC5AuXf616AT5K/Dn02vDcbZnFYPZARMt6/W0ajseLWIUi #
puxPe6s7LTy3x+QU47IutSa+DTpce6NnPG8YEjHsLpIfCSZMXjo7Ai5HGi+od82d #
+l0SVDoj8njBtr0FIBMTJYEuBP1drw4quM7NBK2KBbve+O0iINcUacEVDACMT8S #

iWaZVDRQU7xutqPt8FZ6a3tCPR6OYQsvh5Jyu6MyhKSXg14+W39Z33V0WnnHZhTE #
P9qTjz3/UxTkxVK9IhzZpZqdyu3JWhjUEEJGbDDkvga25s2mAAbydVphYnkBAIC2 #
xZCD9c1zRyyAeeSfWQo8H5IyPIEOtRciLt9C+H2vfBECYyz0w/VDGDDY/AgEG/Hy #
3P5JaPZJjv2OW6SnLrncw/MS+SVkOhORyYUjRLkJn9tz7y4DjD2bBsJjObXUZ4S #
aZLTi8BGbFYAGnWhggMgMIIDHAYJKoZlhvcNAQkGMYIDDTCCAwwCAQEwdzBjMQsw #
CQYDVQQGEwJlVUzEXMBUGA1UEChMORGlnaUNlcnQsIEluYy4xOzA5BgNVBAMTMkRp #
Z2IDZXJ0IFRydXN0ZWQgRzQgUINBNDAA5NiBTSEEyNTYgVGlZVN0YW1waW5nIENB #
AhAFRK/zlJ0IOaa/2z9f5WEWMA0GCWCGSAFIawQCAUAoGkwGAYJKoZlhvcNAQkD #
MQsGCSqGSIb3DQEHAQAcBgkqhkiG9w0BCQUxDxcNMjQwNTE0MDY1ODU4WjAvBgkq #
hkiG9w0BCQQxlgQg3Bf25u0skV7YV8kyqOny0RxKb+Lc+u9Qa/ohRGvxa+IwDQYJ #
KoZlhvcNAQEBBQAEggIAGIYJbugrKx3VpWoeAmOJ91tIVc6dnkw1Pi/T2vgqwnrG #
pjQ3j/4GawHnMJUNAqlbZF8MLLuybOHZO9Y76sg14B7sUiNHXjZKkbNYeYjPrabF #
dn/WjFvifkJoSnrf2aSrK1bThjdY0ZZ3Qz8D9w5vgIKyaTkpfKFHg5w71vUEqFc #
5NI9CRQHJCcjYq7JtDWrt+Ej6/U7eL4doPsCMER27m9t942L661yOUoTAYVqZEO #
oTN0KepwECODggyUWSGATFpxteGOYkimh4dj5PiT1huAqSWRbd56N0GgiTj1cqvi #
FxM1JXZ4kplXVtlbixGWYjYS3FR8sIXd5knOqBQk9My8jAlyNO9ukXhmyzYpfYzj #
1EIU37abmKqZ/Yx7PvX+RGwAgLKdayniab2PLy2yhrI7YrXQgTZC3Yun58CDb+Uz #
2CdaG+opDhRg6/VrG6GjkyS1dUAI3O/qkjg9wDZ0U2UwQiEs132pllViqTY8Djz4 #
FlaQ2jUZuYkCkHy9sgMR5izWGLSxD3SL3Dka3CcA2xyhO/j2axsn5wBI0ILRPL/I #
9adzX5USDrS0qd+ES5B/KXaL938Zf2hG4igDCIyoXxo8Lc5mlq8JspKOgxXUam48 #
atMzo5XpFacHe3Ewx+lvBa1QvjieO+WsiRWMVsDCaXwLjpJPE2jLJQloM9asv+A= # SIG #
End signature block